Confidence- and Dependency-Aware Belief Revision for Structured Clarification Agents

Rania Wehbe
Master 2 in Data Science
Lebanese University
Lebanon
raniawehbe410@gmail.com

Hala Damaj
Master 2 in Data Science
Lebanese University
Lebanon
haladamaj.15@gmail.com

Rayane Berro
Master 2 in Data Science
Lebanese University
Lebanon
berrorayane@gmail.com

***Abstract*—Large language model agents increasingly rely on structured tool calling to perform goal-oriented tasks, but maintaining reliable tool arguments becomes challenging when users revise, weaken, or retract information during multi-turn conversations. This work investigates belief revision for structured clarification agents and proposes a framework combining three mechanisms: a revision ledger for tracking changes to previously established arguments, confidence-aware revision for distinguishing definite and tentative changes, and dependency-aware revalidation for reconsidering arguments affected by a revision. We evaluate the approach on ClarifyBench-Revise, a controlled extension of ClarifyBench containing 216 conversations, including 162 revision-oriented cases. On the revision benchmark, the complete method achieves 100% correctness with no stale values or execution errors, compared with 66.7% for the ledger alone and 83.3% when either confidence awareness or dependency revalidation is added individually. The proposed method also matches the 100% correctness of an Always-Confirm strategy while reducing the average number of clarification questions from 3.73 to 3.40. In a controlled natural-language evaluation using a Gemini-based reader, structured value extraction reaches 100% accuracy and the complete system maintains 100% final correctness. These results indicate that jointly modeling revision history, confidence, and argument dependencies can improve the handling of evolving user intentions while limiting unnecessary clarification.**

***Index Terms*—LLM agents, clarification, belief revision, uncertainty, tool calling, dialogue state tracking**

I. Introduction

Large language model (LLM) agents increasingly interact with external tools and APIs to perform goal-oriented tasks, including through structured function calling [1], [2]. In these settings, an agent must transform natural-language requests into structured tool arguments before deciding whether it has sufficient information to execute a tool. However, real conversations are rarely static: users may provide incomplete information, express uncertainty, revise previously stated preferences, or explicitly retract earlier values. An effective conversational agent must therefore not only identify missing information, but also maintain and revise its beliefs as the conversation evolves.

Recent work has addressed this challenge by explicitly modeling uncertainty during clarification. SAGE (Structured Argument Uncertainty guided Elicitation) represents uncertainty over structured tool arguments rather than relying only on token-level uncertainty [3]. It maintains candidate tool calls as possible hypotheses and uses the Expected Value of Perfect Information (EVPI) together with a redundancy cost to determine whether asking a clarification question is worthwhile. This allows the agent to selectively request information when clarification is expected to improve the quality of the final tool call.

While structured uncertainty provides a principled mechanism for deciding when to clarify, an additional challenge arises when conversational state changes during multi-turn interaction. Prior dialogue-state tracking research has shown that previously established slot values can be difficult to update correctly and that dependencies among slots can affect state tracking [4]–[6]. A previously supplied argument may be corrected, weakened, or explicitly withdrawn, and changing one argument may invalidate another dependent argument. For example, in a travel-booking conversation, a user may replace a previously specified destination, tentatively suggest a new departure airport, or change the departure location in a way that makes the existing destination infeasible. These situations require more than identifying missing values: the agent must determine which information should be replaced, how strongly a revision should be trusted, and whether related arguments must be reconsidered.

To address these challenges, we propose a confidence- and dependency-aware belief revision framework for structured clarification agents. The framework introduces three complementary mechanisms. First, a revision ledger maintains the history of argument updates and supports replacement and explicit retraction of previously supplied values. Second, motivated by prior work on uncertainty-aware clarification [7], confidence-aware revision distinguishes between definite, preferred, and uncertain changes, allowing tentative revisions to trigger clarification rather than being accepted immediately. Third, dependency-aware revalidation, which we refer to as the Domino mechanism, rechecks related arguments when a revision may invalidate previously accepted information. Together, these mechanisms allow the agent to revise its conversational state while selectively asking clarification questions when uncertainty or dependency conflicts remain.

We evaluate the proposed framework using ClarifyBench-Revise, a controlled extension constructed from the Clarify-

Bench benchmark introduced with SAGE [3], to introduce revision-oriented conversational scenarios. The evaluation covers definite revisions, preference-level changes, uncertain revisions, explicit retractions, and dependency-sensitive changes. We compare a SAGE-style re-implementation with successive ablations of the proposed mechanisms and an always-confirm strategy. We additionally evaluate a natural-language setting in which a Gemini-based reader converts user utterances into structured updates. The results show that the full method correctly handles all revision conversations in our controlled benchmark while avoiding stale values and invalid executions. Compared with always confirming every uncertain update, the proposed method achieves the same final correctness with fewer clarification questions.

II. Related Work

A. Structured Tool Use and Dialogue State Tracking

Tool-using conversational agents require a structured representation of information accumulated across multiple dialogue turns. Dialogue state tracking (DST) traditionally represents this information as slot–value pairs that are updated as the conversation progresses. Recent work has connected this formulation more directly to large language model function calling. Li et al. proposed FnCTOD, which reformulates zero-shot dialogue state tracking as function calling, representing dialogue domains as functions and slot values as function arguments [1]. Beyond isolated function calls, ToolTalk evaluates language models in multi-turn conversations that require interactions with external tools, emphasizing the importance of maintaining task-relevant information throughout an evolving dialogue [2].

B. Uncertainty-Aware Clarification

Clarification is particularly important when an agent cannot confidently determine the user's intended tool arguments. SAGE moves uncertainty reasoning from token space to structured argument space by maintaining candidate tool calls as hypotheses and estimating whether resolving an uncertain argument is worth the cost of asking the user [3]. Its clarification policy uses Expected Value of Perfect Information (EVPI) together with a redundancy cost to selectively ask questions rather than clarifying every uncertain parameter. Related work by Deng et al. also studies uncertainty-aware clarification, using belief updates and information gain to identify questions that are expected to reduce uncertainty [7]. These approaches establish the importance of selective clarification under uncertainty. Our work focuses on a complementary setting in which uncertainty arises during the revision of information that has already been established, requiring the agent to decide whether a proposed change should immediately replace the previous value or first be clarified.

C. State Revision and Dependency Modeling

Maintaining a dialogue state requires more than accumulating new information, because previously stored values may later need to be corrected or reconsidered. Liao et al. study dialogue state tracking through incremental reasoning, where the state evolves across dialogue turns and reasoning can incorporate relationships between the current dialogue and previously accumulated information [4]. MoNET further identifies "state momentum," in which dialogue state trackers tend to retain previously predicted slot values even when those values should be updated, and proposes noise-enhanced training to improve state correction [5]. Dependencies among state variables have also been explicitly investigated; Wang et al. model relationships among slots, values, and dialogue context to improve dialogue state tracking [6]. These studies demonstrate that state revision and dependency modeling are important problems, but they do not directly address our combination of explicit user-driven revisions and retractions, revision confidence, and dependency-triggered revalidation within a structured clarification agent. Our framework brings these mechanisms together so that a change to one tool argument can revise its stored belief and, when necessary, reopen related arguments for clarification.

III. Methodology

A. Problem Formulation

We consider a multi-turn tool-calling agent that must maintain a structured state while interacting with a user. Let the tool require a set of arguments $P = \{p_1, \ldots, p_n\}$, where each argument represents a task-specific parameter. In the travel domain used in our experiments, these parameters include the departure airport, destination airport, travel date, and travel class. During a conversation, the user may provide new values for missing arguments or revise values that were established in earlier turns. The agent must maintain the current state, determine whether a revision should be accepted or clarified, and execute the tool only when the resulting arguments are sufficiently resolved and mutually consistent.

At dialogue turn t, we represent the agent's structured belief state as B_t, which contains the currently supported value or values for each tool argument. When the user provides a new observation o_t, the agent updates this state according to

$$B_t = \mathcal{U}(B_{t-1}, o_t),$$

where $\mathcal{U}$ denotes the belief-update mechanism. A central difficulty occurs when o_t conflicts with information already stored in B_{t-1}. Rather than treating every new value as an unconditional replacement, our framework considers the revision history, the confidence associated with the new information, and dependencies between arguments before determining the updated state.

B. SAGE-Style Baseline

As the reference point for our experiments, we implement a simplified SAGE-style baseline inspired by the structured uncertainty formulation of SAGE [3]. The baseline maintains possible values for each tool argument and uses an intersection-based update when new information is received. If a newly observed value is compatible with the current belief, the state can be updated normally. However, when

a user explicitly changes an already established value, the intersection between the previous and new values may become empty. In our implementation, this produces a broken state rather than explicitly representing the new information as a revision of the previous belief.

For example, suppose the current belief for the destination argument is $B_{t-1}(p) = \{\mathrm{LAX}\}$, and the user later changes the destination to JFK. An intersection-based update gives

$$\{\mathrm{LAX}\} \cap \{\mathrm{JFK}\} = \emptyset.$$

In our SAGE-style baseline, this contradiction cannot be interpreted as an explicit replacement of LAX by JFK. This motivates the introduction of a revision mechanism that preserves the history of changes and distinguishes a genuine user correction from ordinary uncertainty.

C. Revision Ledger

o explicitly handle changes in previously established tool arguments, we introduce a revision ledger. Instead of storing only the latest value of each argument, the ledger records the values introduced during the conversation together with their associated support. When a user supplies a value for an unresolved argument, it is added normally. When the user provides a different value for an argument that already has an established value, the event is treated as a revision rather than as an incompatible observation. This allows the agent to preserve the previous value while representing the newly proposed value as the current revision candidate. For a revision from an old value v_{old} to a new value v_{new}, the ledger does not immediately discard the previous value. Instead, the support assigned to the two alternatives depends on the confidence of the revision. A high-confidence correction can strongly favor the new value, whereas a tentative correction preserves meaningful support for the previous value. Explicit retractions are handled separately by removing the retracted value from the active state. This representation therefore supports three operations that are important in evolving conversations: adding new information, revising previously supplied information, and forgetting information that the user explicitly withdraws.

D. Confidence-Aware Revision

Not all user revisions should be treated with the same degree of certainty. We therefore associate each revision with one of three confidence categories: SURE, PREFER, or UNSURE. In our experiments, these categories are assigned heuristic confidence weights of 0.95, 0.70, and 0.50, respectively. These values are experimental design choices rather than calibrated probabilities. They allow the agent to distinguish a definite correction from a preference or a tentative suggestion when updating the revision ledger.

Given a proposed revision, the confidence weight determines how strongly the new value is supported relative to the previous value. In our implementation, SURE revisions receive a confidence of 0.95, which exceeds the execution threshold of 0.90 and can therefore be accepted without additional confirmation when no other unresolved issue remains. PREFER and UNSURE revisions receive lower confidence values and may trigger a clarification question before execution. If the user confirms the proposed value, the corresponding belief is strengthened; if the user rejects it, the agent can return to the previously supported value rather than losing it.

E. Dependency-Aware Revalidation

Revising one tool argument can affect the validity of other arguments that were previously accepted. To address this issue, we introduce dependency-aware revalidation, referred to as the *Domino* mechanism. The method defines dependencies between related tool arguments and checks these relationships whenever one of the arguments is revised. If a revision makes a dependent argument inconsistent or infeasible, that argument is reopened for revalidation instead of being silently retained in the current state. For example, suppose the current state contains `travel_from` = JFK and `travel_to` = LAX. If the user later changes the departure airport, the destination should not automatically be assumed to remain valid. The Domino mechanism checks whether the revised departure and the existing destination still form a valid route according to the dependency information available to the agent. If the combination is invalid, the dependent destination argument is marked for revalidation and the agent requests clarification before execution. This prevents a locally valid revision from producing a globally inconsistent tool call.

F. Full Proposed Method

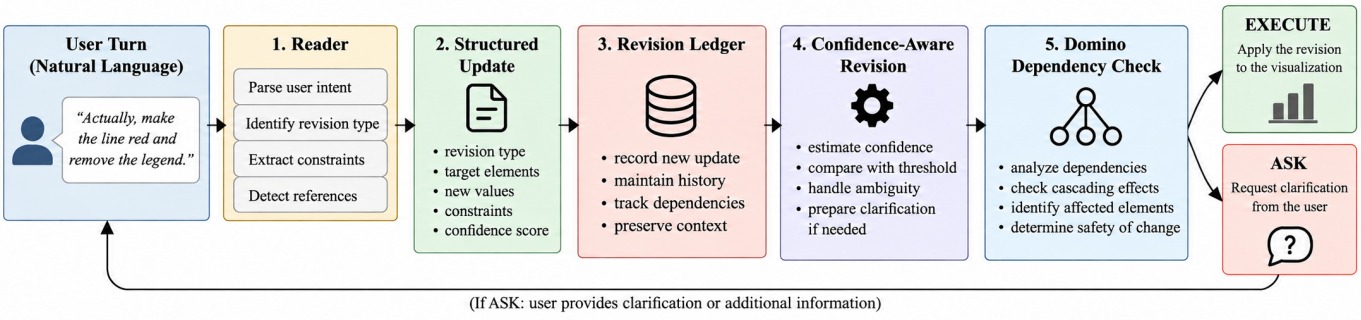
The full proposed method combines the revision ledger, confidence-aware revision, and dependency-aware revalidation within a single belief-update process. For each new user turn, the structured update is first compared with the current state to determine whether it introduces new information, revises an existing value, or retracts previously supplied information. The revision ledger records the change, while the confidence mechanism determines how strongly a revised value should be accepted. The Domino mechanism then checks whether the revision affects any dependent arguments. After these updates, the agent either executes the tool when the state is sufficiently resolved or asks a clarification question when uncertainty or dependency conflicts remain.

Figure 1 illustrates the complete revision-handling pipeline, from the structured user update to the final clarification or execution decision.

IV. Experimental Setup

A. ClarifyBench and ClarifyBench-Revise

We base our evaluation on ClarifyBench, the benchmark introduced with SAGE for evaluating clarification behavior in tool-using LLM agents [3]. To specifically study belief revision, we construct *ClarifyBench-Revise*, a controlled extension derived from real TravelAPI tasks and routes in ClarifyBench. The resulting dataset contains 216 conversations spanning eight scenario types. Of these, 54 are non-revision control conversations and 162 contain revision-oriented interactions used for the main revision evaluation.



Overview of the proposed revision-handling pipeline. A natural-language user turn is processed by the *reader* to extract a structured update, which is recorded in the revision ledger. The system then performs confidence-aware revision and domino dependency checking to decide whether to execute the revision or ask the user for clarification.

Fig. 1. Overview of the proposed revision-handling pipeline.

B. Revision Scenarios

ClarifyBench-Revise contains eight controlled scenario types: *explicit*, *normal*, *revise-sure*, *revise-prefer*, *revise-unsure-new*, *revise-unsure-old*, *retract*, and *domino*. The *explicit* and *normal* scenarios serve as non-revision controls. The remaining six scenarios evaluate different revision behaviors: definite corrections, preference-level changes, uncertain revisions where either the new or previous value is ultimately correct, explicit removal of previously supplied information, and revisions that invalidate a dependent argument. Each scenario type contains 27 conversations, giving 216 conversations in total and 162 revision conversations for the main revision analysis.

C. Baselines and Ablations

We compare six configurations to isolate the contribution of each proposed component. The *SAGE-style baseline* is our simplified re-implementation of structured uncertainty-based clarification and serves as the reference configuration; it should not be interpreted as the official SAGE implementation. *Ledger* adds explicit revision and retraction handling. *Ledger+Confidence* additionally incorporates confidence-aware revision, while *Ledger+Domino* combines the ledger with dependency-aware revalidation. *Always-Confirm* represents a conservative strategy that requests confirmation whenever necessary rather than selectively accepting revisions. Finally, *Ours* combines the revision ledger, confidence-aware revision, and Domino revalidation into the complete proposed method.

D. Evaluation Metrics

We evaluate each method using both task-success and interaction-efficiency measures. *Correctness* indicates whether the final executed tool arguments match the ground-truth state. *Stale value* measures whether an outdated argument remains after a user revision. *Execution error* records cases in which the agent attempts to execute an invalid or inconsistent tool call, while *broken state* captures contradictions that prevent the baseline from maintaining a valid belief state. We additionally report the average number of clarification questions (*Avg. Questions*) required per conversation. Together, these metrics measure not only whether the agent eventually reaches the correct state, but also how safely and efficiently it handles revisions.

E. Natural-Language Evaluation

In addition to the structured evaluation, we evaluate the framework in a controlled natural-language setting. The 162 revision conversations are converted into natural-language utterances, producing 324 user messages. A Gemini-based reader converts each utterance into a structured update containing the affected argument, its value, and a confidence category. The resulting structured update is then passed to the same belief-revision agent used in the main evaluation. This separation allows us to evaluate the proposed revision mechanism while also measuring errors introduced by the natural-language interpretation layer.

V. Results

A. Overall Revision Results

Table I summarizes the performance of all six methods on the 162 revision conversations, while Figure 2 provides a visual comparison of their overall correctness. The results show a clear progression as the proposed revision mechanisms are introduced. Our SAGE-style baseline does not successfully resolve the controlled revision cases, whereas adding the revision ledger increases correctness to 66.7%. Adding either confidence-aware revision or dependency-aware revalidation further improves correctness to 83.3%. The complete method achieves 100% correctness, matching the Always-Confirm strategy while requiring fewer clarification questions on average.

The ablation results highlight the complementary roles of the three proposed mechanisms. The revision ledger alone resolves many direct corrections but remains insufficient for uncertain and dependency-sensitive cases. Adding confidence

TABLE I
OVERALL RESULTS ON 162 REVISION CONVERSATIONS

Method	Correct	Stale	Exec.	Broken	Avg. Q.
SAGE-style baseline	0.0	83.3	0.0	83.3	1.25
Ledger	66.7	33.3	16.7	0.0	2.73
Ledger+Confidence	83.3	16.7	16.7	0.0	3.23
Ledger+Domino	83.3	16.7	0.0	0.0	2.90
Always-Confirm	100.0	0.0	0.0	0.0	3.73
Ours	100.0	0.0	0.0	0.0	3.40

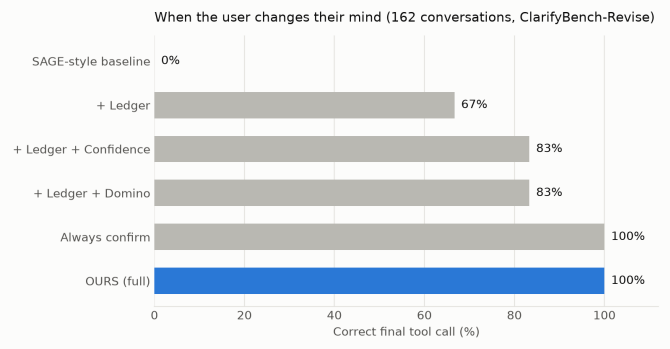


Fig. 2. Overall correctness of the evaluated methods on the 162 revision conversations.

awareness improves the handling of tentative revisions, while adding Domino specifically addresses dependency conflicts and eliminates execution errors caused by invalid dependent arguments. Combining all three mechanisms yields 100% correctness with no stale values, execution errors, or broken states. Although Always-Confirm also achieves 100% correctness, it requires 3.73 clarification questions per conversation compared with 3.40 for our method, indicating that selective revision handling can preserve correctness while reducing unnecessary interaction.

B. Results by Revision Type

To better understand where each component contributes, we analyze correctness separately across the six revision scenario types. Figure 3 shows that the individual ablations solve different subsets of the revision problem. In particular, confidence-aware revision addresses uncertain updates, whereas dependency-aware revalidation addresses Domino cases in which changing one argument affects another. Only the complete method successfully handles all six revision types in our controlled evaluation.

The per-scenario results further demonstrate that the proposed components address different failure modes. The Ledger configuration successfully handles *revise-sure*, *revise-prefer*, and *retract* cases, but fails on uncertain revisions and Domino cases. Adding confidence awareness resolves both types of uncertain revision, but does not address dependency-sensitive changes. Conversely, adding Domino resolves dependency-sensitive cases while still failing when an uncertain revision requires confidence-aware clarification. The full method combines these complementary capabilities and achieves 100% correctness across all six revision categories.

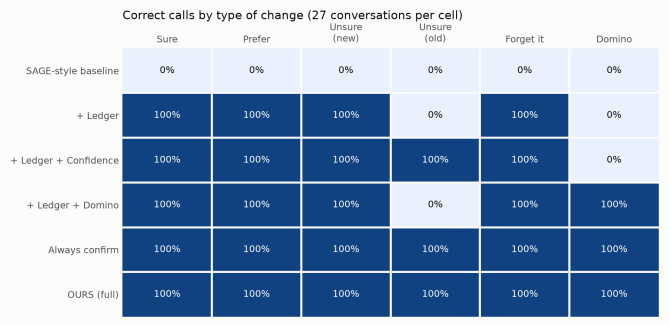


Fig. 3. Correctness by revision type across the evaluated methods.

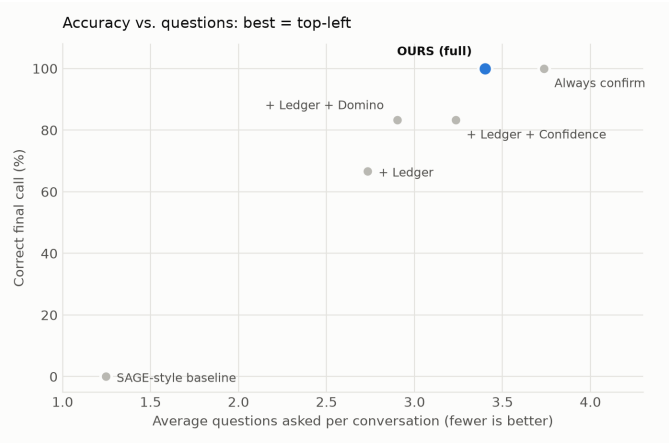


Fig. 4. Trade-off between correctness and the average number of clarification questions on the structured revision benchmark.

C. Accuracy–Clarification Trade-off

Correctness alone does not capture the quality of a clarification strategy, because an agent could obtain high accuracy simply by repeatedly asking the user to confirm information. We therefore examine the trade-off between final correctness and the number of clarification questions. As shown in Figure 4, both Always-Confirm and our complete method achieve 100% correctness on the structured revision benchmark. However, our method requires an average of 3.40 questions per conversation compared with 3.73 for Always-Confirm, corresponding to approximately 8.8% fewer clarification questions.

D. Natural-Language and Gemini Reader Results

We next evaluate whether the proposed method remains effective when revisions are expressed in natural language rather than provided directly as structured updates. Across the 324 controlled natural-language utterances, the Gemini-based reader achieves 100% structured value extraction accuracy and 99.1% full-turn accuracy. Only three reader errors occur, all involving the confidence label: PREFER is classified as SURE. No argument values are extracted incorrectly.

When the Gemini-extracted updates are passed to the clarification agents, the same overall pattern observed in the structured evaluation is preserved. Our complete method achieves 100% final correctness with no stale values or execution errors

and requires an average of 3.38 clarification questions per conversation. Always-Confirm also achieves 100% correctness but requires 3.73 questions on average. The three confidence-label errors made by the reader do not change the final correctness of our method on this controlled benchmark. However, because these errors are few and the natural-language utterances are controlled, this result should not be interpreted as evidence of general robustness to arbitrary natural-language parsing errors.

E. Sensitivity Analysis

We additionally examine the sensitivity of the proposed method to the heuristic confidence weights used for revision handling. Across the first five tested confidence configurations, the complete method maintains 100% correctness, with an average of approximately 3.20 clarification questions per conversation. However, when the confidence assigned to UNSURE revisions is increased to 0.92, above the execution threshold of 0.90, correctness decreases to 85.7%. This occurs because an uncertain revision can then be treated as sufficiently confident for execution without first requesting confirmation.

VI. Discussion

The experimental results indicate that robust belief revision requires more than a single mechanism. The revision ledger provides the foundation for replacing or retracting previously established information, but it cannot by itself determine how tentative revisions should be treated or whether a change invalidates related arguments. Confidence-aware revision and dependency-aware revalidation address these two limitations in complementary ways. This is reflected in the ablation results: each mechanism improves a different subset of revision scenarios, while their combination is required to achieve complete correctness across all six controlled revision types.

The comparison with Always-Confirm also highlights the importance of clarification efficiency. Asking for confirmation whenever uncertainty is present can provide strong protection against incorrect revisions, but it increases the interaction burden on the user. Our complete method reaches the same 100% correctness as Always-Confirm on the structured revision benchmark while reducing the average number of clarification questions from 3.73 to 3.40. This suggests that revision history, confidence, and dependency information can be used together to avoid some unnecessary confirmations without sacrificing correctness in the evaluated setting.

The natural-language evaluation provides an additional test of the framework when structured revisions must first be inferred from user utterances. The Gemini reader extracts all argument values correctly and achieves 99.1% full-turn accuracy, with only three confidence-label errors. Despite these errors, the complete method retains 100% final correctness on the controlled natural-language benchmark. This result suggests that separating natural-language interpretation from structured belief revision is a practical design choice: the reader converts an utterance into a structured update, while the revision mechanism reasons over the resulting state. Nevertheless, broader evaluation with more diverse and naturally occurring language is required before drawing conclusions about robustness to general parsing errors.

VII. Limitations

This study has several limitations. First, our SAGE-style baseline is a simplified re-implementation rather than the official SAGE agent, and the reported baseline results should therefore not be interpreted as performance results for the original SAGE system. Second, ClarifyBench-Revise is a programmatically constructed benchmark derived from ClarifyBench TravelAPI tasks and routes. Although this controlled design allows specific revision behaviors to be isolated, it does not capture the full diversity and ambiguity of naturally occurring conversations. Third, the current dependency rules are manually specified for the travel domain, particularly relationships between departure and destination arguments, and their generalization to other domains remains to be evaluated.

Fourth, the confidence values used for SURE, PREFER, and UNSURE are heuristic experimental weights rather than probabilities learned or calibrated from user data. The sensitivity analysis shows that performance can degrade when these values interact poorly with the execution threshold, motivating future work on learned or calibrated confidence estimation. Finally, the main structured evaluation uses oracle-style structured turns and simulated ground-truth responses to clarification questions, while the natural-language evaluation uses controlled generated utterances. Future work should evaluate the framework with real users, more diverse linguistic expressions, additional tool domains, and automatically learned dependency structures.

VIII. Conclusion

This work investigated belief revision in structured clarification agents when users modify, weaken, or retract information during multi-turn interactions. We proposed a framework combining a revision ledger, confidence-aware revision, and dependency-aware revalidation to maintain a consistent tool-argument state as user intentions evolve. On the 162 revision conversations in our controlled ClarifyBench-Revise benchmark, the complete method achieved 100% correctness with no stale values or execution errors, while requiring fewer clarification questions than an Always-Confirm strategy. The controlled natural-language evaluation further showed that the method maintained 100% final correctness when structured updates were produced by a Gemini-based reader. These results demonstrate the potential value of explicitly modeling revision history, confidence, and argument dependencies in clarification agents. Future work should evaluate these mechanisms with real user interactions, additional domains, learned confidence estimates, and automatically discovered dependency structures.

References

[1] Z. Li, Z. Z. Chen, M. Ross, P. Huber, S. Moon, Z. Lin, X. L. Dong, A. Sagar, X. Yan, and P. A. Crook, "Large language models as zero-shot dialogue state tracker through function calling," in *Proceedings of the 62nd Annual Meeting of the Association for Computational*

Linguistics (Volume 1: Long Papers). Bangkok, Thailand: Association for Computational Linguistics, 2024, pp. 8688–8704.

[2] N. Farn and R. Shin, "Tooltalk: Evaluating tool-usage in a conversational setting," *arXiv preprint arXiv:2311.10775*, 2023.

[3] M. Suri, P. Mathur, N. Lipka, F. Dernoncourt, R. A. Rossi, and D. Manocha, "Structured uncertainty guided clarification for LLM agents," in *Findings of the Association for Computational Linguistics: ACL 2026*, 2026, pp. 40 811–40 838.

[4] L. Liao, L. H. Long, Y. Ma, W. Lei, and T.-S. Chua, "Dialogue state tracking with incremental reasoning," *Transactions of the Association for Computational Linguistics*, vol. 9, pp. 557–569, 2021.

[5] H. Zhang, J. Bao, H. Sun, Y. Wu, W. Li, S. Cui, and X. He, "MoNET: Tackle state momentum via noise-enhanced training for dialogue state tracking," in *Findings of the Association for Computational Linguistics: ACL 2023*. Toronto, Canada: Association for Computational Linguistics, 2023, pp. 520–534.

[6] Q. Wang, Y. Cao, P. Li, Y. Fu, Z. Lin, and L. Guo, "Slot dependency modeling for zero-shot cross-domain dialogue state tracking," in *Proceedings of the 29th International Conference on Computational Linguistics*. Gyeongju, Republic of Korea: International Committee on Computational Linguistics, 2022, pp. 510–520.

[7] M. Deng, Z. Li, X. Li, T. Zhu, Y. Zhao, Z. Guo, and W. Wang, "Uncertainty-aware clarification in LLM agents with information gain," in *Proceedings of the 43rd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 306. PMLR, 2026, pp. 24 083–24 102.